

claude-windows / c7679509-ee2b-4443-b240-0b3b1b1a7291

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c7679509-ee2b-4443-b240-0b3b1b1a7291\subagents\workflows\wf\_a2b97cff-dcd\agent-ae0680c9645abdbf1.jsonl`
- SHA-256: `5c5fa73b46fdec16c6daca7c42a738d612abb4076a216f64af8bafd6aa6157da`
- Source modified: `2026-06-21T13:37:40+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf\_a2b97cff-dcd`
- session_id: `c7679509-ee2b-4443-b240-0b3b1b1a7291`

Transcript

1. user (2026-06-21T13:34:48.461Z)

PROJECT: a local, AI-powered badminton/racket-sports HIGHLIGHT INDEXER + rally detector.
Engine repo (PRIMARY): C:/Users/avidu/Projects/badminton-highlight-indexer (Python, FastAPI + SQLite + ffmpeg + GPU CV/AI; paid Gemini = cloud AI).
Sibling repos: C:/Users/avidu/Projects/khelsutra (the web/ React SPA + site/ marketing), C:/Users/avidu/Projects/rally-annotator (VLC plugin, MIT), C:/Users/avidu/Projects/sports-data-collector, C:/Users/avidu/Projects/sports-obsreport.

GOAL of the plan we are building: bake the engine into a DOWNLOADABLE, "batteries-included" Python APP that a user can download and run WITHOUT sourcing the repo (no git clone / build-from-source). It must spin up a local WEB SERVICE that runs (a) INFERENCE (video -> rallies -> highlight reel), (b) TRAINING (BYO model on their own labelled data), and (c) RALLY ANNOTATION if needed -- runnable BOTH locally and against CLOUD services. Preserve every already-completed end-to-end CUJ.

KEY FACTS the orchestrator already verified:

- pyproject.toml: hatchling build, namespace package (no backend/__init__.py), packages=["backend","training"], console script: indexer = backend.main:main. requires-python >=3.10.
- torch/torchvision are DELIBERATELY NOT in dependencies -- their CUDA/CPU wheels need --index-url (PEP621 cannot express). Optional extras: gpu(empty, docs-only), auth(PyJWT), reporting(git+ssh obsreport), storage-s3(boto3), storage-gcs(google-cloud-storage). google-genai is a hard dep.
- Server today: python -m backend.main --server --port 8000 . Worker: python -m backend.worker {infer|train} . CLI single-shot: python -m backend.main --video-path ... --segmenter {gemini|yolo_hybrid|wasb_hybrid|motion}.
- THE HARD PACKAGING PROBLEM: native WASB shuttle model runs in a SEPARATE Python env (conda, torch 1.11+cu113) while the main engine uses torch 2.6+cu124. ffmpeg/ffprobe are system deps. Model weights (wasb_badminton_best.pth.tar) live in a GCS bucket.
- Decoupled-compute SEAM already built (M1-M8 merged, default-OFF): deployment.profile + compute_target on AppConfig; backend/config/presets.py (PROFILE_PRESETS: truly-local / cloud-serving / cpu-mock); backend/storage/* (local/gcs/s3/gdrive StorageBackend registry); backend/compute/* (ComputeBackend registry, CloudRunCompute dispatches a Cloud Run Job); backend/config/startup.py per-profile checks; backend/billing.py cost ledger.
- The web SPA lives in a DIFFERENT repo (khelsutra/web, Vite/React/TS/Tailwind, i18n en+kn+hi). The engine has a likely-DEAD backend/api/static. Bundling the UI is a cross-repo concern.
- Licensing guardrail (NON-NEGOTIABLE): every dependency -- code, data, AND weights -- must be MIT/Apache-2.0/BSD only. Paid LLMs = inference/serving only, never training data. Any NEW packaging dependency must also be license-clean.
- A prior docs/DESKTOP_APP_PLAN.md exists but was SUPERSEDED by owner decision D10: deliver a "self-hosted webserver + UI the user spins up" (NOT a desktop app).

khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md is the live appliance design.
Always ground claims in file:line or a doc path. Read the actual files; do not assume.

AREA: verification + licensing constraints on packaging. READ: run_all_tests.py, .github/workflows/ci.yml (the CI lanes incl torch/coverage floor), pyproject dev extras, and any license-gate code (grep gate_verdict, no_paid_llm_training_labels, license). Answer: how the packaged app would be tested (offline mocked suite? smoke test?); the coverage floor + CI lanes that any packaging change must keep green; the commercial-clean licensing rule and which candidate packaging tools (PyInstaller GPL? conda Anaconda-TOS? Nuitka? Docker? micromamba?) are license-compatible vs problematic.

2. assistant (2026-06-21T13:34:51.482Z)

I'll investigate the verification and licensing constraints on packaging. Let me start by pulling the latest and reading the key files.

3. user (2026-06-21T13:34:54.527Z)

fatal: Cannot fast-forward to multiple branches.

4. assistant (2026-06-21T13:34:57.048Z)

Let me work around the branch issue and read the key files.

5. user (2026-06-21T13:34:58.844Z)

feat/serving-m9-auth
43eccc1 feat(serving): M8 ? per-job cost ledger + versioned pricebook + /cost API (default-OFF) (#288)

6. user (2026-06-21T13:34:58.970Z)

```
1      #!/usr/bin/env python
2      """Run the FULL test suite for the badminton-highlight-indexer.
3
4      A single, discoverable entry point so you (or a reviewer / CI) can verify the
5      whole suite before opening a PR:
6
7          python run_all_tests.py          # full suite, quiet
8          python run_all_tests.py -v       # verbose (passthrough to pytest)
9          python run_all_tests.py -k config # filter by keyword (passthrough)
10         python run_all_tests.py --cov=backend --cov-report=term-missing # coverage
11
12     Anything after the script name is passed straight through to pytest.
13
14     Notes
15     -----
16     - The suite is fully mocked (no GPU, no network, no WSL) and runs offline.
17     - The observability **reporting** tests skip cleanly if the optional ``obsreport``
18       package isn't installed (they use ``pytest.importorskip``). To exercise them too:
19         pip install -e ../sports-obsreport --no-build-isolation
20     - Exit code is pytest's own, so this is CI / pre-PR friendly (0 = all green).
21     """
22     from __future__ import annotations
23
24     import importlib.util
25     import subprocess
26     import sys
27
28
29     def _installed(module: str) -> bool:
30         try:
31             return importlib.util.find_spec(module) is not None
32         except (ImportError, ValueError):
33             return False
34
35
```

```

36     def main() -> int:
37         args = list(sys.argv[1:])
38         # Default to quiet unless the caller chose a verbosity.
39         if not any(a in ("-q", "-v", "-vv", "-vvv", "--quiet", "--verbose") for a in args):
40             args = ["-q", *args]
41         cmd = [sys.executable, "-m", "pytest", *args]
42
43         obs = "available" if _installed("obsreport") else "NOT installed ? reporting tests
will SKIP"
44         bar = "=" * 72
45         print(bar)
46         print(" Full test suite: " + " ".join(cmd))
47         print(f" obsreport (optional reporting dep): {obs}")
48         print(bar, flush=True)
49
50         rc = subprocess.run(cmd).returncode
51
52         print(bar)
53         print(" RESULT: " + ("PASS - all tests green" if rc == 0 else f"FAIL - pytest exit
{rc}"))
54         print(bar)
55         return rc
56
57
58     if __name__ == "__main__":
59         raise SystemExit(main())
60

```

7. assistant (2026-06-21T13:35:01.856Z)

The pull failed due to multiple tracking branches, which is fine?I'll proceed read-only. Let me read the CI workflow and pyproject in parallel.

8. user (2026-06-21T13:35:02.342Z)

```

1     name: CI
2
3     on:
4         push:
5             branches: [master]
6         pull_request:
7
8     permissions:
9         contents: read
10
11     concurrency:
12         group: ci-${{ github.ref }}
13         cancel-in-progress: true
14
15     jobs:
16         # Lint gate (ruff.toml at repo root): pyflakes/pycodestyle + bugbear.
17         # Also the repo-wide syntax guard ? ruff reports invalid-syntax on every
18         # tracked .py, including scripts/ and training/, which the backend-only
19         # py_compile smoke sweep does not reach.
20         lint:
21             name: Ruff lint
22             runs-on: ubuntu-latest
23             steps:
24                 - uses: actions/checkout@v6
25                 - uses: actions/setup-python@v6
26                   with:
27                       python-version: "3.13"
28                 - name: Install ruff
29                   run: python -m pip install ruff
30                 - name: Ruff check

```

```

31         run: ruff check . --output-format github
32
33     # Type gate (mypy.ini at repo root): lenient green baseline with an explicit
34     # per-module quarantine list ? ratchet by removing entries as modules clean
35     # up. Typed core deps are installed so mypy sees their real signatures
36     # (without them ignore_missing_imports would Any-stub half the checks).
37     typecheck:
38         name: Mypy (lenient baseline)
39         runs-on: ubuntu-latest
40         steps:
41             - uses: actions/checkout@v6
42             - uses: actions/setup-python@v6
43               with:
44                 python-version: "3.13"
45             - name: Install mypy + typed core deps
46               run: python -m pip install mypy pydantic fastapi uvicorn python-dotenv google-
genai numpy
47             - name: Mypy
48               run: mypy backend training
49
50     # Fast guard against the 7fbbeb0 class of breakage: a SyntaxError or
51     # module-level import error shipping while the suite stays green.
52     # Deliberately does NOT install cv2/torch/numpy ? the smoke tests must hold
53     # on machines without the GPU stack (test_import_smoke stubs what's absent).
54     import-smoke:
55         name: Syntax sweep + import smoke (no GPU stack)
56         runs-on: ubuntu-latest
57         steps:
58             - uses: actions/checkout@v6
59             - uses: actions/setup-python@v6
60               with:
61                 python-version: "3.13"
62             - name: Install minimal deps
63               run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-
genai
64             - name: Run import smoke tests
65               run: python -m pytest tests/test_import_smoke.py -v
66
67     full-suite:
68         name: Full test suite (offline, mocked)
69         runs-on: ubuntu-latest
70         steps:
71             - uses: actions/checkout@v6
72             - uses: actions/setup-python@v6
73               with:
74                 python-version: "3.13"
75                 cache: pip
76             - name: System libs for opencv-python
77               run: |
78                 sudo apt-get update
79                 sudo apt-get install -y libgl1 libglu1-mesa-dev || sudo apt-get install -y
libgl1 libglu1-mesa-dev
80             - name: Install CPU-only torch (no CUDA wheels on a CPU runner)
81               # torch/torchvision are NOT in pyproject deps ? their CPU/CUDA wheels
82               # need an out-of-band index that PEP 621 can't express. Keep installing
83               # them separately here, before the editable install pulls the rest.
84               run: python -m pip install torch torchvision --index-url
https://download.pytorch.org/whl/cpu
85             - name: Install package (editable) + dev tooling
86               # Editable PEP 621 install: runtime deps + the `dev` group
87               # (pytest, pytest-cov, httpx for the fastapi TestClient, ruff, mypy).
88               run: python -m pip install -e .[dev]
89             # obsreport (private, soft dep) is absent here: reporting tests skip via
90             # pytest.importorskip ? run_all_tests.py prints which. The coverage floor
91             # is therefore calibrated against THIS lane's number (local runs with

```

```

92         # obsreport installed measure a little higher).
93         # Floor calibrated 2026-06-11 against this lane's measured 72% (local
94         # with obsreport: 73%). Ratchet it UP as coverage grows; never down
95         # without an owner decision.
96         - name: Run full suite (with coverage floor)
97           run: python run_all_tests.py --cov=backend --cov=training --cov-report=term
--cov-fail-under=70
98
99         # Cross-OS determinism guard for the committed golden regression fixtures
100        # (docs/GOLDEN_REGRESSION_FIXTURES.md). The CONTROL replay path is pure-stdlib
101        # (only numpy, no cv2/torch), so this is a FAST, focused job ? it does NOT run
102        # the full suite x3 OSes. It replays the Windows/Linux/macOS runner against the
103        # committed (parse-compared) snapshots, catching any cross-OS float drift beyond
104        # the 1e-6 tolerance before it can flake a contributor's PR on another machine.
105        golden-crossos:
106          name: Golden fixtures cross-OS (${{ matrix.os }})
107          strategy:
108            fail-fast: false
109            matrix:
110              os: [ubuntu-latest, windows-latest, macos-latest]
111          runs-on: ${{ matrix.os }}
112          steps:
113            - uses: actions/checkout@v6
114            - uses: actions/setup-python@v6
115              with:
116                python-version: "3.13"
117            - name: Install minimal deps (no GPU stack ? the CONTROL replay needs only numpy)
118              run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-
genai numpy
119            - name: Golden-fixtures characterization (cross-OS determinism guard)
120              # Also runs the freeze-real / time-origin-reset / M2 refusal-gate tests
(synthetic /
121              # tmp_path only ? no GPU/network) so the reset metric-invariance (fix #2) and
the
122              # de-attribution paths get cross-OS coverage and surface any win/mac float
drift.
123              run: |
124                python -m backend.eval.golden_fixtures --check
125                python -m pytest tests/test_golden_fixtures.py
tests/test_golden_real_fixtures.py -q
126

```

9. user (2026-06-21T13:35:02.817Z)

```

1         # PEP 621 packaging for the badminton/sports highlight indexer (DEFERRED #7).
2         #
3         # Build backend: hatchling. `backend/` is an intentional NAMESPACE package
4         # (no `backend/__init__.py`; mypy.ini sets explicit_package_bases for the same
5         # reason), so the wheel target packages are listed EXPLICITLY below ? hatchling's
6         # auto-detection would otherwise miss the un-__init__'d `backend` root.
7         #
8         # torch/torchvision are deliberately NOT listed in [project.dependencies]:
9         # their CPU/CUDA wheels need an out-of-band index (--index-url / --extra-index-url),
10        # which PEP 621 cannot express. Install them separately ? see [project.optional-
dependencies]
11        # `gpu` (documentation only) and the README. CI installs CPU torch on its own line.
12        [build-system]
13        requires = ["hatchling"]
14        build-backend = "hatchling.build"
15
16        [project]
17        name = "badminton-highlight-indexer"
18        version = "0.1.0"
19        description = "AI-assisted rally segmentation and highlight indexing for racket sports."
20        readme = "README.md"

```

```

21     requires-python = ">=3.10"
22     license = { text = "MIT" }
23     authors = [{ name = "Avi Dullu", email = "avi.dullu@gmail.com" }]
24     keywords = ["badminton", "sports", "video", "highlights", "rally", "segmentation"]
25
26     # Runtime deps mirror requirements.txt, EXCEPT torch/torchvision (see header).
27     dependencies = [
28         "fastapi>=0.111.0",
29         "uvicorn>=0.30.1",
30         "opencv-python>=4.9.0.80",
31         "numpy>=1.26.4",
32         "pydantic>=2.7.1",
33         "jinja2>=3.1.4",
34         "python-dotenv>=1.0.0",
35         "google-genai>=2.4.0",
36         "supervision>=0.21.0,<0.30.0",
37     ]
38
39     [project.optional-dependencies]
40     # Test + lint/type tooling ? matches the CI lanes (.github/workflows/ci.yml).
41     dev = [
42         "pytest",
43         "pytest-cov",
44         "httpx",          # fastapi TestClient transport
45         "ruff",
46         "mypy",
47         "PyJWT[crypto]>=2.8", # M9 edge-auth: Cf-Access JWT verify (RS256) ? tests + CI
48     ]
49
50     # M9 edge-auth (Cloudflare Access). DEFAULT-OFF: only needed when
51     security.edge_auth_enabled=True.
52     # `jwt` is lazy-imported, so the base runtime works without this; install it for
53     hosted/multi-tenant.
54     auth = ["PyJWT[crypto]>=2.8"]
55
56     # Per-video observability reports (SOFT dependency: the pipeline degrades to a
57     # no-op if absent). Pinned to its git tag per BACKLOG. The private repo is
58     # fetched over SSH; if you lack access, simply omit this group ? reporting
59     # tests skip via pytest.importorskip. For local dev against a sibling checkout:
60     # pip install -e ../sports-obsreport --no-build-isolation
61     reporting = [
62         "obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.3",
63     ]
64
65     # `gpu` is DOCUMENTATION ONLY ? it is intentionally empty. torch/torchvision
66     # CUDA wheels require `--index-url`/`--extra-index-url`, which PEP 621 cannot
67     # express, so they are NOT declarable here. Install them out-of-band instead:
68     # # CUDA 12.4 box:
69     # pip install torch>=2.12.0 torchvision>=0.27.0 --index-url
70     # https://download.pytorch.org/whl/cu124
71     # # CPU-only box (also what CI uses):
72     # pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
73     gpu = []
74
75     # Storage backends (docs/DECOUPLED_COMPUTE/). The default `local` backend needs NONE of
76     # these. Cloud SDKs are imported lazily, so `import backend.storage` works without them;
77     # install the extra for the bucket you use. (boto3's S3 client covers AWS + Cloudflare
78     R2 +
79     # DO Spaces + MinIO via endpoint_url.) `gdrive` needs NO extra ? it reads a locally
80     # MOUNTED Google Drive (Google Drive for Desktop) as plain files; see
81     backend/storage/gdrive.py.
82     storage-s3 = ["boto3>=1.34"]
83     storage-gcs = ["google-cloud-storage>=2.16"]
84
85     [project.scripts]

```

```

81     # `indexer --server`, `indexer --help`, etc. -> backend/main.py::main()
82     indexer = "backend.main:main"
83
84     [tool.hatch.metadata]
85     # The `reporting` extra pins obsreport to a git tag (a PEP 508 *direct
86     # reference*); hatchling requires opting in to allow those in metadata.
87     allow-direct-references = true
88
89     [tool.hatch.build.targets.wheel]
90     # `backend` is a namespace package (no __init__.py); list both import roots
91     # explicitly so an editable/wheel install exposes `import backend` and
92     # `import training` regardless of cwd.
93     packages = ["backend", "training"]
94

```

10. assistant (2026-06-21T13:35:05.423Z)

Now let me look at the license-gate code and related testing infrastructure.

11. user (2026-06-21T13:35:09.535Z)

Found 6 files
 backend\pipeline\segmenters\yolo_hybrid.py
 backend\eval\golden_real_fixtures.py
 backend\eval\golden_fixtures.py
 backend\pipeline\detectors\person_detector.py
 backend\assets\fonts\LICENSE-Apache-2.0.txt
 backend\assets\fonts\README.md

12. user (2026-06-21T13:35:09.742Z)

No files found

13. assistant (2026-06-21T13:35:13.138Z)

The license-gate is about model weights / training data provenance, not a runtime "gate_verdict" function. Let me search more broadly for the licensing gate concept and the de-attribution / no-paid-LLM-training paths mentioned in CI.

14. user (2026-06-21T13:35:14.708Z)

Found 16 files
 docs\COMPUTE_DECOUPLED_SERVING\README.md
 docs\data_pipeline\GOLDEN_DATA_SHARING.md
 docs\RALLY_DETECTION_QUALITY_REPORT.md
 docs\POST_166_WORK_ITEMS_MILESTONES_AND_RISK_ASSESSMENT.md
 docs\OWNED_MODEL_IMPLEMENTATION_PLAN.md
 docs\CODE_MAP.md
 docs\DESKTOP_APP_PLAN.md
 docs\GOLDEN_REGRESSION_FIXTURES.md
 docs\README.md
 training\gen0\harness.py
 tests\test_training_gen0.py
 backend\eval\golden_real_fixtures.py
 backend\eval\golden_fixtures.py
 tests\test_golden_real_fixtures.py
 .github\workflows\ci.yml
 artifacts\rally_tal_gen0\0.1.0\manifest.json

15. user (2026-06-21T13:35:15.714Z)

Found 89 files
 backend\main.py
 backend\infrastructure\database.py

backend\config\models.py
tests\test_billing.py
backend\api\job_worker.py
tests\test_job_worker_loop.py
tests\test_async_jobs.py
tests\test_database.py
tests\test_api_endpoints.py
tests\test_compute_backend.py
backend\worker__main__.py
backend\compute\cloud_run.py
tests\test_profile_parity.py
backend\pipeline\detectors\native_wasb_runner.py
tests\test_server_entrypoint.py
backend\pipeline\segmenters\gemini.py
backend\pipeline\segmenters\yolo_hybrid.py
backend\pipeline\segmenters\trajectory_hybrid.py
backend\utils\paths.py
tests\test_config_deployment.py
training\gen0\harness.py
tests\test_worker.py
tests\test_training_gen0.py
backend\storage\base.py
artifacts\so.py
backend\pipeline\segmenters\fusion_hybrid.py
backend\pipeline\segmenters\wasb_hybrid.py
backend\pipeline\detectors\wasb_infer.py
tests\test_stub_runner.py
backend\pipeline\stitcher\compiler.py
backend\pipeline\detectors\stub_runner.py
tests\test_court_gate.py
backend\eval\rally_seg_eval.py
backend\pipeline\detectors\court_gate.py
tests\test_rally_seg_eval.py
backend\providers\gemini.py
tools\generate_highlights.py
tests\test_rally_seg_eval_report.py
tests\test_motion_segmenter.py
tests\test_job_worker_module_seam.py
tests\test_golden_fixtures_split.py
tests\test_experiment.py
backend\pipeline\segmenters\motion.py
backend\eval\golden_real_fixtures.py
backend\eval\golden_fixtures.py
backend\eval\fusion_compare.py
backend\eval\experiment.py
backend\eval\ablation.py
tests\test_served_gate_a_regression.py
backend\api\uploads.py
tests\test_nightly_regression.py
scripts\run_phase3_eval.py
scripts\nightly_regression.py
tests\test_golden_real_fixtures.py
tests\test_golden_fixtures.py
backend\eval\tolerance_metrics.py
backend\eval\segmentation_metrics.py
tests\test_tracknet_hybrid.py
backend\pipeline\detectors\base.py
backend\eval\per_user_regression.py
backend\eval\per_user_eval.py
backend\eval\per_user_gate.py
tests\test_per_user_gate.py
tests\test_fusion_golden_telemetry.py
backend\eval\batch.py
backend\utils\run_telemetry.py
backend\pipeline\detectors\person_detector.py


```

backend\utils\logging_setup.py
tests\test_logging_setup.py
tests\test_watermark.py
backend\pipeline\audio\hit_detector.py
tests\test_reporting.py
backend\reporting\__init__.py
backend\pipeline\validators\scene_cut.py
tests\test_run_signals.py
backend\reporting\run_signals.py
tests\test_reporting_harvest.py
run_regression.py
backend\eval\regression.py
tests\test_reporting_spec_faq.py
tests\test_hit_detector.py
tests\test_automated_provider.py
backend\eval\export_wasb_segments.py
backend\eval\gemini_refine.py
backend\annotations\automated_provider.py
tests\test_annotation_providers.py
backend\annotations\file_provider.py
backend\annotations\base.py
backend\sports\badminton.py

```

16. assistant (2026-06-21T13:35:18.720Z)

Let me look at the licensing-gate code in the golden fixtures / training harness (where "no_paid_llm_training" and de-attribution appear), and the M2 refusal-gate mentioned in CI.

17. user (2026-06-21T13:35:21.458Z)

```

1: """Golden Regression Fixtures (P1 + M2) ? de-attribution / minimization + the refusal gate.
5: into a committable, de-attributed Tier-0 fixture, reusing M1's ``replay_fixture`` for the
  snapshot.
6: (docs/GOLDEN_REGRESSION_FIXTURES.md §4c anti-attribution, §5 threat model, §7 row P1, §6
  honest
12: defined here (``freeze_real_fixture``, ``RefusalError``, ``_scan_forbidden``) at the bottom
  of that
53: # P1 ? De-attribution + minimization: OWNED real trajectory+labels ? committable Tier-0
55: # §4c anti-attribution, §5 threat model, §7 row P1, §6 honest limits.)
59: class RefusalError(ValueError):
60:     """The provenance/biometric refusal gate (M2) rejected an unsafe freeze.
63:     names *why* the unsafe public-fixture path was refused. The refusal is the code-enforced
64:     provenance gate (§4c "provenance hard-fail gate", §7 ?): the owner-recorded / minors-
  excluded
65:     / license flags are human-asserted, but the gate makes the unsafe path impossible to
  reach
86:     "provenance", "provenance.json",
91:     """POSITIVE privacy assertion (§4c, red-team ?): raise if ``text`` leaks anything
  attributable.
102:         raise RefusalError(
108:         raise RefusalError(
110:             f"video_id shape and an attribution back-channel; never write it (§4c/§5).")
125:         raise RefusalError(
162:     license: str,
172:     """Turn an OWNED, minors-free real trajectory + golden labels into a committable, de-
  attributed
175:     The refusal gate (M2) makes the unsafe path impossible to reach silently: a fixture is
  emitted
176:     ONLY when ``minors_free is True`` AND ``owned is True`` AND ``license`` is a non-blank
  string,
178:     player/event name is refused, red-team #1). The committed bytes carry NO filename /
  video_id /
181:     ``kind="cleared_real"`` provenance block. A positive privacy assertion re-reads the
  written
182:     ``expected.json`` + ``provenance.json`` and refuses if anything attributable slipped in.

```

```

184:  ``source_note`` is **accepted but NOT persisted** ? operator free-text is an attribution
187:  committed ``provenance.json`` (red-team #1). It is still scanned defensively and the
flag is
190:  NOTE ($6 honest limits): the random surrogate + reset timeline make the fixture non-
attributable
192:  attribution does not cure provenance/licensing; this sits BEHIND counsel sign-off (P2)
and the
196:  after the privacy scan passes ? a refused / failed freeze leaves NO partial fixture dir
201:  # --- (a) REFUSAL GATE (M2) ? the code-enforced provenance hard-fail ($4c / $7 ?)
-----
209:  if not (isinstance(license, str) and license.strip()):
210:      reasons.append("license must be a non-blank string (per-record provenance tag ? $4c
"
211:                    "'provenance hard-fail gate')")
212:  # label is committed verbatim into provenance ? it MUST be a strict slug, never operator
free
218:      f"name) is an attribution leak; got {label!r} ($4c metadata strip, red-team
#1)")
220:      raise RefusalError(
221:          "REFUSED to freeze a real Tier-0 fixture ? unsafe provenance (de-attribution
does NOT "
222:          "cure provenance/licensing; this gate sits behind counsel sign-off, $6/P2):\n -
"
261:  # start). Refuse loudly up-front so the partial-write path is never even reached.
264:      raise RefusalError(

```

[Showing results with pagination = limit: 40]

18. user (2026-06-21T13:35:22.477Z)

```

52:# tarball-build time is the off-box provenance fallback (see _git_sha +
deploy/gcp/README.md).
64:  Guards every provenance source so a stray value (``unknown\\n``, a build label) can never
be
79:  Steps 2-4 make a remote tarball run provenance-stamped WITHOUT changing the git-present
path.
175:  reasons.append("WASB checkpoint provenance C3 UNRESOLVED ? commercial ship blocked
regardless of F1")
234:      "deps_licenses": "numpy/scipy (BSD) + repo-internal code only (Gen-0)",
235:      "wasb_checkpoint_provenance": "C3-UNRESOLVED (ship-blocking; see
COMMERCIALIZATION C3)",
236:      "no_paid_llm_training_labels": True,

```

19. assistant (2026-06-21T13:35:24.955Z)

This is the key licensing-gate code. Let me read the relevant sections of both files in more detail.

20. user (2026-06-21T13:35:25.701Z)

```

160      reasons.append(f"LOVO mean {eval_res['mean_f1']:.3f} <= floor
{floor_mean:.3f}")
161      per = floor.get("per_video", {})
162      paired = [(r["f1"], per[r["video"]]) for r in eval_res["per_video"] if
r["video"] in per]
163      if paired:
164          wins, n, p = sign_test([lv for lv, _ in paired], [fv for _, fv in paired])
165          sign = {"wins": wins, "n": n, "p_one_sided": round(p, 4)}
166          if p >= 0.05:
167              reasons.append(f"paired sign test not significant ({wins}/{n} wins,
p={p:.3f}) "
168                             f"? grow the corpus before claiming superiority")
169      else:
170          reasons.append("no floor baseline provided ? nothing to beat")

```

```

171
172     # Provisional blocker (per owner review on #170; see #172 for calibration).
173     # (The number itself is not the blocker; calibration tracked in #172.)
174     reasons.append("ship-gate F1@tIoU target NOT YET CALIBRATED ? provisional; see issue
#172")
175     reasons.append("WASB checkpoint provenance C3 UNRESOLVED ? commercial ship blocked
regardless of F1")
176     # ship remains False until C3 resolved + other gates (by design).
177     return {"ship": False, "floor_passed": floor_passed, "sign_test": sign, "blockers":
reasons}
178
179
180 def run(manifest_path: str, out_dir: str, version: str,
181         floor_path: Optional[str] = None) -> dict:
182     with open(manifest_path, encoding="utf-8") as f:
183         specs = json.load(f)
184         floor = None
185         if floor_path:
186             with open(floor_path, encoding="utf-8") as f:
187                 floor = json.load(f)
188
189         # 1) Honest LOVO eval (the number that matters).
190         eval_res = lovo_run(specs)
191
192         # 2) Final model on all videos.
193         clf, thr = train_final(specs)
194
195         # 3) Gate (pre-flight).
196         verdict = gate_verdict(eval_res, floor)
197
198         # 4) Artifact bundle.
199         bundle_dir = os.path.join(out_dir, version)
200         os.makedirs(bundle_dir, exist_ok=True)
201         weights_path = os.path.join(bundle_dir, "weights.json")
202         payload = clf.to_dict()
203         payload["calibration"] = {"threshold": thr, **CALIBRATION_DEFAULTS}
204         payload["feature_schema_version"] = FEATURE_SCHEMA["version"]
205         with open(weights_path, "w", encoding="utf-8") as f:
206             json.dump(payload, f, indent=2)
207
208         manifest = {
209             "artifact": {
210                 "name": "rally_tal_gen0", "version": version,
211                 "created_at": time.strftime("%Y-%m-%dT%H:%M:%S%z"),
212                 "git_sha": _git_sha(),
213                 "weights_file": "weights.json", "weights_sha256": _sha256(weights_path),
214             },
215             "feature_schema": FEATURE_SCHEMA,
216             "calibration": {"threshold": thr, **CALIBRATION_DEFAULTS,
217                             "note": "threshold tuned train-side on the full corpus; "
218                                     "honest generalization = the LOVO eval below"},
219             "training_lineage": {
220                 "corpus_manifest": manifest_path,
221                 "split_protocol": "leave-one-video-out by match",
222                 "label_source": "human (rally-annotator golden CSVs)",
223                 "videos": [{ "name": s["name"], "labels": s["labels"],
224                             "gt_hash": gt_hash(load_gt_intervals(s["labels"],
strict=False)),
225                             "fps": s["fps"]} for s in specs],
226             },
227             "eval": {
228                 "protocol": "LOVO, F1@tIoU=0.5 segment-level + per-frame AUC",
229                 "per_video": eval_res["per_video"],
230                 "mean_f1": eval_res["mean_f1"],
231                 "floor": floor,

```

```

232         },
233         "attestation": {
234             "deps_licenses": "numpy/scipy (BSD) + repo-internal code only (Gen-0)",
235             "wasb_checkpoint_provenance": "C3-UNRESOLVED (ship-blocking; see
COMMERCIALIZATION C3)",
236             "no_paid_llm_training_labels": True,
237             "consent_ledger": "pending-M0.1 (owner-attested: owned footage, all-adult
sessions)",
238         },
239         "gate": verdict,
240     }
241     manifest_out = os.path.join(bundle_dir, "manifest.json")
242     with open(manifest_out, "w", encoding="utf-8") as f:
243         json.dump(manifest, f, indent=2)
244
245     print(f"\n=== Gen-0 artifact bundle: {bundle_dir} ===")
246     print(f"weights : {weights_path}")
247     print(f"manifest: {manifest_out}")
248     print(f"GATE      : ship={verdict['ship']} floor_passed={verdict['floor_passed']} "
249           f"sign_test={verdict['sign_test']}")

```

21. assistant (2026-06-21T13:35:29.484Z)

Now let me check the test count, the dev tooling versions, and look at any existing packaging/Docker artifacts. Let me also examine the README for the documented install/run flow.

22. user (2026-06-21T13:35:34.853Z)

```

112
---DOCKER---
./deploy/cloudrun/Dockerfile
---DEPLOY---
cloudrun
digitalocean
gcp
---CONDA/ENV---
./claude/worktrees/agent-a771ff1d85e18f2ed/requirements.txt
./claude/worktrees/agent-a771ff1d85e18f2ed/training/requirements-train.txt
./claude/worktrees/bhi-execpolicy/requirements.txt
./claude/worktrees/bhi-execpolicy/training/requirements-train.txt
./claude/worktrees/boxhill17/requirements.txt
./claude/worktrees/boxhill17/training/requirements-train.txt
./claude/worktrees/gap-doc/requirements.txt
./claude/worktrees/gap-doc/training/requirements-train.txt
./claude/worktrees/hardchop/requirements.txt
./claude/worktrees/hardchop/training/requirements-train.txt
./claude/worktrees/pr-stitch/requirements.txt
./claude/worktrees/pr-stitch/training/requirements-train.txt
./claude/worktrees/recdoc/requirements.txt
./claude/worktrees/recdoc/training/requirements-train.txt
./claude/worktrees/resweep/requirements.txt
./claude/worktrees/resweep/training/requirements-train.txt
./claude/worktrees/tier1-windowing/requirements.txt
./claude/worktrees/tier1-windowing/training/requirements-train.txt
./claude/worktrees/w2-safeguard/requirements.txt
./claude/worktrees/w2-safeguard/training/requirements-train.txt
./requirements.txt
./training/requirements-train.txt

```

23. user (2026-06-21T13:35:38.864Z)

```

1     fastapi>=0.111.0
2     uvicorn>=0.30.1
3     opencv-python>=4.9.0.80

```

```

4      numpy>=1.26.4
5      pydantic>=2.7.1
6      jinja2>=3.1.4
7      python-dotenv>=1.0.0
8      google-genai>=2.4.0
9      torch>=2.12.0
10     torchvision>=0.27.0
11     supervision>=0.21.0,<0.30.0
12
13     # Per-video observability reports (SOFT dependency: the pipeline degrades to a
14     # no-op if absent). Shared, permissively-licensed report toolkit. Pin a tag once
15     # the private repo is published, e.g.:
16     #   obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.3
17     # For local development install it editable from a sibling checkout:
18     #   pip install -e ../sports-obsreport --no-build-isolation
19

```

24. user (2026-06-21T13:35:39.656Z)

```

1      # Training-only dependencies (ADR-008): NEVER merge these into the root
requirements.txt.
2      # The serving/control plane must stay importable without the training stack.
3      #
4      # Gen-0 (the LogReg/TAL-head seed) needs nothing beyond the root requirements
5      # (numpy/scipy ship there already). Heavier Gen-0 candidates and Gen-2 land here:
6      #
7      # ActionFormer / ASFormer are research repos (MIT), not pip packages ? clone + install
8      # per their READMEs into the training env (WSL conda recommended), e.g.:
9      #   git clone https://github.com/happyharrycn/actionformer_release # MIT
10     #   git clone https://github.com/ChinaYi/ASFormer # MIT
11     #
12     # Gen-2 (when greenlit; ADR-008 split trigger):
13     #   ms-swift # Apache-2.0 ? training forge (video SFT/GRPO)
14     #   torch / transformers pins per ms-swift's matrix, in ITS OWN env ? not here-ported.
15

```

25. user (2026-06-21T13:35:39.944Z)

```

1      # Cloud Run GPU **Job** image for the rally worker (COMPUTE_DECOUPLED_SERVING M6/M7,
ADR-012/D16).
2      #
3      # Runs `python -m backend.worker infer ?` INLINE on an L4 (detect + window + stitch co-
located so
4      # CPU/wall + egress are metered into one job ? D4). Built + pushed + verified by the
owner on the
5      # M7 real run (the $100/?10k budget, D17); CI does NOT build this (a CUDA image is too
heavy) ? the
6      # dispatch shim is proven with a fake client (tests/test_compute_backend.py).
7      #
8      # Two python stacks, deliberately separate (a framework conflict otherwise ? same split
as the
9      # native GPU box, deploy/digitalocean/gpu_setup.sh):
10     #   * the MAIN app env (this repo + ffmpeg) drives the pipeline + stitching;
11     #   * a conda `wasb` env (py3.8 + torch 1.11.0+cu113) runs the WASB shuttle detector as
a subprocess
12     #       (the native runner shells out to $WASB_PYTHON), so the app's torch never co-
installs with it.
13     #
14     # ? WASB-C3: the pretrained badminton weights are academic-data-trained ? provenance NOT
cleared for
15     #   commercial sharing. They are NOT baked into this image; stage them at runtime (a
bucket fetch or a
16     #   mount) and point $WASB_WEIGHTS at them. Resolve WASB-C3 before any public, non-owner
serving.
17

```

```

18     FROM nvidia/cuda:12.4.1-runtime-ubuntu22.04
19
20     ENV DEBIAN_FRONTEND=noninteractive \
21         PYTHONUNBUFFERED=1 \
22         PIP_NO_CACHE_DIR=1 \
23         CONDA_DIR=/opt/conda \
24         WASB_REPO_DIR=/opt/models/WASB-SBDT
25
26     # ffmpeg (stitch/decode) + DejaVu fonts (the watermark filter) + git/curl for the WASB
env build.
27     RUN apt-get update && apt-get install -y --no-install-recommends \
28         ffmpeg fonts-dejavu-core git curl ca-certificates build-essential \
29         python3 python3-pip python3-venv \
30         && rm -rf /var/lib/apt/lists/*
31
32     # --- WASB detector env (mirrors deploy/digitalocean/gpu_setup.sh ? the verified
torch-1.11 stack) ---
33     RUN curl -sSL https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh -o
/tmp/mc.sh \
34         && bash /tmp/mc.sh -b -p "$CONDA_DIR" && rm -f /tmp/mc.sh \
35         && "$CONDA_DIR/bin/conda" tos accept --override-channels --channel
https://repo.anaconda.com/pkgs/main || true \
36         && "$CONDA_DIR/bin/conda" tos accept --override-channels --channel
https://repo.anaconda.com/pkgs/r || true \
37         && "$CONDA_DIR/bin/conda" create -y -q -n wasb python=3.8 \
38         && git clone --depth 1 https://github.com/nttcom/WASB-SBDT.git "$WASB_REPO_DIR" \
39         && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
40             torch==1.11.0+cu113 torchvision==0.12.0+cu113 --extra-index-url
https://download.pytorch.org/whl/cu113 \
41         && ( [ -f "$WASB_REPO_DIR/requirements.txt" ] && "$CONDA_DIR/envs/wasb/bin/python"
-m pip install -q -r "$WASB_REPO_DIR/requirements.txt" || true ) \
42         && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
43             hydra-core omegaconf pandas opencv-python-headless pillow scipy tqdm pyyaml
matplotlib "numpy<1.24"
44
45     # --- Main application env ---
46     WORKDIR /app
47     COPY pyproject.toml README.md ./
48     COPY backend ./backend
49     COPY training ./training
50     RUN python3 -m pip install --upgrade pip && python3 -m pip install .
51
52     # The native runner finds the WASB env + repo by env (no `conda activate` needed);
weights come in at
53     # runtime (WASB-C3). detector_impl=native = the Linux-native WASB runner; cuda stays the
default.
54     ENV WASB_PYTHON=/opt/conda/envs/wasb/bin/python \
55         WASB_REPO_DIR=/opt/models/WASB-SBDT \
56         WASB_WEIGHTS=/opt/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar \
57         RALLY_DETECTOR_IMPL=native
58
59     # The dispatcher (CloudRunCompute) supplies the per-execution args:
60     # infer --video-uri gs:/// --out-uri gs:/// --done-uri gs:/// --segmenter wasb_hybrid
--set ?
61     ENTRYPOINT ["python3", "-m", "backend.worker"]
62

```

26. assistant (2026-06-21T13:35:44.548Z)

This Dockerfile is highly relevant?it already documents the conda+Anaconda-ToS approach for the WASB env. Note line 35: `conda tos accept` ? this confirms the Anaconda ToS friction is real and already worked around in the cloud image. Let me check the import smoke test and confirm how the suite handles missing GPU stack, plus check for any existing packaging plan docs.

27. user (2026-06-21T13:35:45.980Z)

```

1      """Import/syntax smoke tests for the backend package.
2
3      Guards against a SyntaxError (or module-level import error) shipping while the
4      suite stays green: most of the suite imports individual backend modules, so a
5      broken file that nothing imports (e.g. the stitcher compiler, which only
6      backend.main pulls in) used to go unnoticed.
7
8      Both tests are offline and dependency-light: the syntax sweep never imports
9      anything, and the backend.main import runs in a subprocess with any missing
10     heavy deps (cv2/torch/numpy) stubbed, so they also pass in environments
11     without the GPU stack.
12     """
13
14     import py_compile
15     import subprocess
16     import sys
17     from pathlib import Path
18
19     import pytest
20
21     REPO_ROOT = Path(__file__).resolve().parents[1]
22
23
24     def test_backend_package_has_no_syntax_errors():
25         """Every .py file under backend/ must byte-compile."""
26         failures = []
27         for path in sorted((REPO_ROOT / "backend").rglob("*.py")):
28             try:
29                 py_compile.compile(str(path), doraise=True)
30             except py_compile.PyCompileError as exc:
31                 failures.append(f"{path.relative_to(REPO_ROOT)}: {exc.msg}")
32         assert not failures, "Syntax errors in backend/:\n" + "\n".join(failures)
33
34
35     def test_backend_main_imports():
36         """backend.main (FastAPI app, /api/compile) must be importable.
37
38         Runs in a subprocess so stubbing missing heavy deps cannot leak into this
39         pytest process; cwd=repo root puts the package on sys.path.
40         """
41         prelude = (
42             "import importlib.util, sys\n"
43             "from unittest import mock\n"
44             "for name in ('cv2', 'torch', 'numpy'):\n"
45             "    if importlib.util.find_spec(name) is None:\n"
46             "        sys.modules[name] = mock.MagicMock()\n"
47             "import backend.main\n"
48         )
49         result = subprocess.run(
50             [sys.executable, "-c", prelude],
51             cwd=str(REPO_ROOT),
52             capture_output=True,
53             text=True,
54         )
55         assert result.returncode == 0, (
56             "import backend.main failed:\n" + result.stderr.strip()
57         )
58
59
60     def test_serving_never_imports_training():

```

28. user (2026-06-21T13:35:47.324Z)

Exit code 2
 ---CONFTEST---

29. assistant (2026-06-21T13:35:50.995Z)

No conftest.py. Let me check the rest of import_smoke (the serving-never-imports-training test is relevant to packaging boundaries) and check how torch is referenced/mockd in tests, plus look at the DESKTOP_APP_PLAN and LOCAL_APPLIANCE docs briefly.

30. user (2026-06-21T13:35:52.413Z)

```
60     def test_serving_never_imports_training():
61         """THE IMPORT WALL (ADR-008): `import backend.main` must load no training.* module.
62
63         Serving consumes versioned model ARTIFACTS, never training code. This is the
64         executable boundary that lets training live in-repo until the Gen-2 split.
65         """
66         prelude = (
67             "import importlib.util, sys\n"
68             "from unittest import mock\n"
69             "for name in ('cv2', 'torch', 'numpy'):\n"
70             "    if importlib.util.find_spec(name) is None:\n"
71             "        sys.modules[name] = mock.MagicMock()\n"
72             "import backend.main\n"
73             "bad = sorted(m for m in sys.modules\n"
74             "                if m == 'training' or m.startswith('training.'))\n"
75             "assert not bad, f'serving imported training code: {bad}\n"
76             "print('WALL_OK')\n"
77         )
78         result = subprocess.run(
79             [sys.executable, "-c", prelude],
80             cwd=str(REPO_ROOT),
81             capture_output=True,
82             text=True,
83         )
84         assert result.returncode == 0 and "WALL_OK" in result.stdout, (
85             "import wall breached or import failed:\n" + result.stderr.strip()
86         )
87
88
89     def test_backend_no_static_training_imports():
90         """Static sweep: no backend/ source file may import training.* (catches lazy imports
91         inside functions, which the runtime wall test above cannot see)."""
92         offenders = []
93         for path in sorted((REPO_ROOT / "backend").rglob("*.py")):
94             text = path.read_text(encoding="utf-8", errors="replace")
95             for lineno, line in enumerate(text.splitlines(), start=1):
96                 stripped = line.strip()
97                 if stripped.startswith(("import training", "from training")):
98                     offenders.append(f"{path.relative_to(REPO_ROOT)}:{lineno}: {stripped}")
99         assert not offenders, "backend/ imports training code:\n" + "\n".join(offenders)
100
101
102     def test_featurizer_is_pure_numpy_scipy():
103         """backend.features (the shared train==serve featurizer) must import WITHOUT
104         torch/cv2 ? blocking them outright, not just stubbing when absent.
105
106         Requires numpy/scipy on the host (the featurizer's only deps); on the deliberately
107         minimal no-GPU-stack CI lane this skips ? the full-suite lane (which has torch
108         installed) is where blocking torch is a meaningful assertion anyway.
109         """
110         pytest.importorskip("numpy")
111         pytest.importorskip("scipy")
112         prelude = (
113             "import sys\n"
114             "class _Block:\n"
115             "    def find_module(self, name, path=None):\n"
116             "        if name in ('torch', 'cv2') or name.startswith(('torch.', 'cv2.')):\n"
```



```

117         "            return self\n"
118         "    def load_module(self, name):\n"
119         "        raise ImportError(f'{name} is blocked: backend.features must be pure\n"
numpy/scipy')\n"

```

31. user (2026-06-21T13:35:55.227Z)

===license texts in repo===

===DESKTOP_APP_PLAN head===

Cross-Platform Desktop App ? "Plug in your card, get your highlights"

```

> **Status:** `DRAFT` ? `owner-gated` (scoping doc only; **nothing is built**) . **Owner:**
avidullu . **Created:** 2026-06-18 . **Last updated:** 2026-06-18
> **Lifecycle:** `DRAFT` ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when DONE)
> **Tracking anchors:** $7 progress tracker is the source of truth; indexed in `docs/README.md`;
pointer in memory `current-state` (and `docs/NEXT_STEPS.md` once the owner greenlights P0).
> **Relation to existing docs:** **peer-of**
[`PLATFORM_ARCHITECTURE.md`](PLATFORM_ARCHITECTURE.md) (this is the **`LocalDesktop`
ComputeBackend** made into a product) . **realizes the L0 tier** of
[`VIDEO_LOCALITY_MODEL.md`](VIDEO_LOCALITY_MODEL.md) . **consumes**
[`OWNED_MODEL_IMPLEMENTATION_PLAN.md`](OWNED_MODEL_IMPLEMENTATION_PLAN.md) (the smaller/export-
clean model is one answer to the compute crux) . bound by
[`COMMERCIALIZATION.md`](COMMERCIALIZATION.md) licensing guardrails.
> **Honesty note:** every claim is tagged `[verified]` (checked against code/measured),
`[researched]` (needs sign-off), or `[design]` (proposed). **The entire build is `[design]`** ?
this doc scopes; it does not ship. Not legal advice.

```

0. TL;DR

Package the mature local pipeline (WASB shuttle detector + improved windowing + ffmpeg stitch, already runnable fully-offline via [tools/generate_highlights.py --skip-ai-handoff](../tools/generate_highlights.py)) as a ****cross-platform desktop app**** for ****non-technical recreational players****: plug in the camera's USB/SD card, click once, get back ****only the highlight reels**** ? fully on-device, ****no upload****, ****no copy of the GBs of 4K originals****. This is the product surface that fits the "local, cheap, efficient" thesis and the L0 ("nothing leaves the machine") tier of `VIDEO\_LOCALITY\_MODEL.md`.

****The make-or-break unknown is compute on a typical enthusiast laptop**** ? which often has ****no discrete GPU****. WASB runs at ****~3.4x real-time on a laptop RTX 2070 Super**** `[verified, measured]`; on CPU it is far slower, and Apple Silicon has ****no CUDA**** at all. ****P0 is a feasibility spike that measures CPU and Apple-MPS throughput for a ~10-min clip before any app is built.**** If CPU-only is unacceptable on commodity hardware, the path needs a smaller/faster owned model (ties `OWNED\_MODEL\_IMPLEMENTATION\_PLAN`) or a tiered "fast-if-GPU, slow-but-works-if-CPU" design. ****Everything past P0 is gated on that result.****

1. Problem & goal

****Why now / the user.**** Target users are ****recreational sports enthusiasts, not tech-savvy****. Their match videos are ****huge**** (GBs of 4K). The cloud-upload product surface fails them on every axis: slow on a typical Indian uplink (`VIDEO\_LOCALITY\_MODEL.md` \$0: 10 GB ? ~1 h at 20 Mbps, ~4.5 h at 5 Mbps), bandwidth-costly, privacy-fraught, and it bloats both our storage and their drives. The thing the customer actually wants back is ****kilobytes of timestamps ? a highlight reel**** (`VIDEO\_LOCALITY\_MODEL.md` \$1: ****"timestamps are the product; the video is dead weight"***).

****The concrete outcome ("good" looks like this):****

1. User plugs the GoPro/phone SD card or camera USB into their laptop.
2. The app detects the card, lists the match videos on it.
3. User clicks ****Generate Highlights****.
4. The app makes an on-device proxy, runs WASB detection + windowing + stitches the reel ? ****100% locally****.
5. The reel(s) land in a user-chosen folder. ****The originals are never copied off the card and**

never bloat any drive.**

No account, no upload, no cloud dependency in the default path. Gemini stays an **optional** "better brain if you want it" toggle ? never required, never bundled as a hard dependency, never a training source (`COMMERCIALIZATION` C4).

Read to get current: `VIDEO\_LOCALITY\_MODEL.md` (the L0/locality framing + OQ5 packaging seed), `PLATFORM\_ARCHITECTURE.md` §3.2 (the ComputeBackend seam), [`backend/pipeline/detectors/base.py`] (`./backend/pipeline/detectors/base.py`) (the detector ABC that already anticipates a Linux-native runner), `tools/generate\_highlights.py` (the local entrypoint this app wraps).

2. Decisions locked

```
| # | Decision | Source / date | Implication |
|---|-----|-----|-----|
| D1 | Default path is 100% on-device, no upload, no Gemini. The reel is produced from the
local pipeline ( `--skip-ai-handoff` ). | This doc · 2026-06-18; matches `VIDEO_LOCALITY_MODEL` L0
| The app must never require a network call. Gemini is an opt-in toggle only. |
| D2 | Never copy the originals. The app reads source video in place (off the card / a
chosen folder) and writes only reels + a tiny report to a user folder. | This doc; mirrors
`generate_highlights.py` non-destructive `_atomic_publish` discipline | No "import library" that
duplicates 10 GB files. Detection runs on a proxy; stitch reads the original in place. |
| D3 | Drop the WSL2 dependency; run the detector natively per-OS. | This doc; enabled by
the `DetectorRunner` ABC `[verified]` | Windows-native (no WSL), Linux+NVIDIA (CUDA directly),
macOS (MPS or CPU). The WSL adapter stays for the current dev box but is not the shipped
path. |
| D4 | Commercial-clean licensing for everything bundled: MIT / Apache-2.0 / BSD only (code,
weights). ffmpeg ships as an LGPL build (no GPL-only encoders like x264). |
`COMMERCIALIZATION` (ratified 2026-06-09); `VIDEO_LOCALITY_MODEL` OQ5 ("ffmpeg LGPL build =
licensing-clean") | WASB-SBDT weights are MIT `[verified]` (see `tracknet-wasb-wsl2-decision`).
The stitch must use openh264 (BSD) or OS hardware encoders, not statically-linked x264. See
§3.4. |
| D5 | Reuse the existing pipeline verbatim; add only the shell + a native runner + device-
IO. | `backend/pipeline/detectors/base.py` reuse contract `[verified]` | No re-architecture of
windowing/stitch. New code = native runner, USB/SD detection, app shell, packaging. |
| D6 | Feasibility is gated on P0. No app shell, no packaging work starts until the compute
spike returns a verdict. | This doc; `decision-rigor-norm` (NUMBERS INFORM, FOUNDATIONS DECIDE)
| P0 is the only un-gated deliverable; P1?P3 are owner-greenlit after P0 numbers. |
===kheIsutra appliance doc?===
/c/Users/avidu/Projects/kheIsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md
```

32. assistant (2026-06-21T13:35:59.820Z)

The DESKTOP_APP_PLAN.md is highly relevant?it already locked D4 (ffmpeg LGPL build, openh264/BSD, no x264) for licensing-clean packaging. Let me read the ffmpeg/encoder licensing section (§3.4) and check the LOCAL_APPLIANCE doc for packaging tool decisions. Let me also confirm whether `scipy` is actually a declared dependency (the featurizer test imports it but it's not in pyproject).

33. user (2026-06-21T13:36:02.301Z)

```
38:| D4 | Commercial-clean licensing for everything bundled: MIT / Apache-2.0 / BSD only
(code, weights). ffmpeg ships as an LGPL build (no GPL-only encoders like x264). |
`COMMERCIALIZATION` (ratified 2026-06-09); `VIDEO_LOCALITY_MODEL` OQ5 ("ffmpeg LGPL build =
licensing-clean") | WASB-SBDT weights are MIT `[verified]` (see `tracknet-wasb-wsl2-decision`).
The stitch must use openh264 (BSD) or OS hardware encoders, not statically-linked x264. See
§3.4. |
45:- WASB on a discrete laptop GPU is acceptable `[verified, measured]`: on the owner's
RTX 2070 Super Max-Q (8 GB), WASB inference is the pipeline driver at ~3.2?3.4× real-
time (?3.4 GPU-min per footage-min, ~0.055 s/frame), peak VRAM ~5.7?6.0 GB; proxy (NVENC)
~0.2× RT; 4K stitch (CPU libx264) ~16 s/rally; end-to-end ?5× clip length. Source: `current-
state.md` `COST_SCALING.md` checkpoint (6 Boxhill clips, 2026-06-18).
51:[Omitted long matching line]
```

63:- **Backend packaging:** the Python/FastAPI backend bundles via **PyInstaller** or a frozen venv as a local "sidecar." Packaging was already modernized ? **PEP 621** `pyproject.toml` + an `indexer` console entry point`` shipped (`DEFERRED_HARDENING_PLAN` #7, DONE, PR #167`) `[verified]`` ? so a CLI MVP has a real head start.

64:- **ffmpeg licensing** (a genuine landmine): **ffmpeg** is LGPL/GPL. The current stitch path uses **libx264** (GPL). Redistributing that inside an installer would impose GPL on the bundle.

Mitigation (D4): ship an **LGPL ffmpeg build** using **openh264** (BSD) or the OS hardware encoder (NVENC / Apple **VideoToolbox** / Linux **VAAPI**) ? the desktop app should prefer hardware H.264 anyway (faster, no CPU stitch tax).

91:| **Trajectory?** rally windowing | **REUSE** `[verified]`` | `tracknet_runner.trajectory_to_action_windows` (model-agnostic staticmethod) |`

99:| **Packaging + code-signing per OS** | **NEW** `[design]`` | **PyInstaller/MSIX** (Win), **.dmg+notarize** (mac), **AppImage/Flatpak** (Linux) ? **P3** |

111:| **ffmpeg GPL contamination via libx264 in the installer** | high (legal) | **D4: LGPL build + openh264/hardware encoders only.** |

133:| **P3** | **Packaging + code-signing installers** per OS (Win **MSIX/Authenticode**, mac **.dmg + Developer-ID notarization**, Linux **AppImage/Flatpak**); **LGPL ffmpeg bundling**; **MIT/Apache/BSD audit** of the bundle. | **P2** | Yes | ? | ? |

152:- [] **P3** signed/notarized installers exist for the greenlit OSes; the bundle passes an **MIT/Apache/BSD (+ LGPL-ffmpeg)** license audit.

159:**External** (to validate at **P0/P2/P3**): **PyTorch MPS backend** · **onnxruntime** (CPU/CoreML/DirectML execution providers) · **Tauri vs Electron sizing** · **ffmpeg LGPL build / openh264** · **Apple Developer-ID notarization** · **Windows Authenticode/MSIX**.

34. user (2026-06-21T13:36:03.849Z)

```
---scipy usage---
backend/eval/rally_seq_proto.py
backend/eval/tolerance_metrics.py
backend/features/rally_seq.py
```

35. assistant (2026-06-21T13:36:08.457Z)

Important finding: `scipy`` is imported by `backend/features/rally_seq.py`` (a serving-side featurizer) but is NOT declared in `pyproject.toml` dependencies. This is a packaging hole. The **DESKTOP_APP_PLAN** already names **PyInstaller** as the candidate and flags the **ffmpeg/libx264 GPL** landmine. Now let me check the live **LOCAL_APPLIANCE** doc (the superseding design) for packaging-tool decisions, and verify `scipy` comes in transitively.

36. user (2026-06-21T13:36:11.427Z)

12:[Omitted long matching line]

21:**Headline CUJ** (owner's words): a **hosted website** where I pick an input ? **a Google Drive file**, a **bucket file** (`s3://`gs://``), or a **local upload** ? it lands in the **bucket** (**DO Spaces / GCS**) or locally ? I **generate highlights** on a **GPU-enabled host** running the real e2e ? I **pick rallies**, watch, and download ? **completely from the UI.**

23:**Outcome** ("good"): an operator signs in, picks a source, watches it index, curates rallies, downloads a reel ? on the **live CPU droplet today** (**Gemini/stub**), and on a **GPU host later** (native **WASB**) with **no UI fork**.

49:- **Absent** (**net-new**): `GET /api/capabilities``, `GET /api/videos`` (only single-row `get_video(id)``), `GET /api/reels/{video_id}``, any source/proxy streaming endpoint, any **Dockerfile**, any per-video **cost ledger** `[verified ? grep returns none]``.

96:| **GPU host ? native WASB** (in-flight) | `wasb+`detector_impl=native` ? NativeWasbRunner` (in-proc torch) |` a **GPU host ? DO GPU droplet / GCE GPU VM / Cloud Run+GPU**; `WASB_WEIGHTS` |` **REAL**, on-device | partially wired (fusion builds it, trajectory refuses); **owner byte-parity gate**; **Cloud Run/GCE** need a containerized worker ? no **Dockerfile** exists, and **WASB's torch-1.11 conda env** makes the image non-trivial |

97:| **Owner Win+WSL ? auto** | `wasb+`detector_impl=auto` (WSL2+conda) |` **Windows+WSL2+GPU** | **REAL** (today) | **Windows-only transport**; the working real-GPU path for owner verification |

103:**No engine auth**, by design; the gate is external. **Invariant** (all three must hold): **engine** bound to `127.0.0.1`` + firewall closes `8000`` + edge gate up. **Go-live checklist** `[HOSTING_PLAN.md:102-108]``: **Access/basicauth ON**; **rotate the chat-shared GEMINI\_API\_KEY**; set `security.allowed_video_root``; **rate-limit**; **per-user quota**.

119:- **Structured logs.** New endpoints/screens emit structured (**JSON-able**) logs at **entry /**

result / error** ? not just "it ran". The SPA logs client-side errors + failed fetches with the same correlation id.

126:| Risk | Today | Mitigation |

135:| No Dockerfile | runbook-only reproducibility | feasibility-check a worker image before promising Cloud Run/GCE |

137:| **Provisioning-credential leak** | a DO API token / cloud creds could leak if pasted in chat or committed (cf. the **already-rotated Spaces key** exposed in chat, `DECOUPLED\_COMPUTE/HANDOFF.md`) | authenticate the CLI **on-box** (`doctl auth init` interactive / `gcloud` already authed) so the secret never enters chat or the repo; least-privilege token, revoke after; creds 0600, never committed; **creating paid GPU infra is a confirm-first action every time** |

139:## 7. Honest limits ? what this does NOT do

140:[Omitted long matching line]

143:**CUJs:** **CUJ-1** ingest (Drive/bucket/local) ? index ? reel ? download (core); **CUJ-2** re-open a past match after reload; **CUJ-3** pick-by-query (deterministic filters + honesty-hiding); **CUJ-4** settings/compute placement (truthful capabilities).

150:- **L5 ? Owner hardware e2e (handed off).** Native WASB **byte-parity** on a real GPU box; real-4K **golden-corpus** runs (Gemini + native); Modern-Standby/`cv2`-vs-`ffmpeg` behavior; a multi-GB browser upload. Reported back into S9, stated plainly as **not verified** in the offline suite.

163:| T6 | S3 embed **RallyPicker** + compile + download | T4 | no | ? | ? |

37. user (2026-06-21T13:36:14.320Z)

```
36:    "supervision>=0.21.0,<0.30.0",
--- is scipy transitive via supervision? ---
Requires: defusedxml, matplotlib, numpy, opencv-python, pillow, pydeprecate, pyyaml, requests,
scipy, tqdm
--- scipy in installed env? ---
scipy 1.17.1
```

38. assistant (2026-06-21T13:36:19.497Z)

Good ? `scipy` does come in transitively via `supervision` (it's in supervision's `Requires`). So it's available in practice but not explicitly declared, which is a packaging fragility (an upstream supervision change could drop it). The LOCAL_APPLIANCE doc confirms "no Dockerfile exists" as a known risk and that the WASB torch-1.11 conda env makes the image non-trivial.

Let me do a final check on the coverage measurement and confirm the exact test count and whether there's a license-scanning tool in CI. Let me also verify the `auth` extra and the obsreport git+ssh dependency (a packaging concern for a downloadable app).

39. user (2026-06-21T13:36:22.472Z)

```
===test count===
110
===any pip-licenses/license check in CI or scripts===
(none if blank)
===ruff.toml + mypy.ini exist===
mypy.ini
ruff.toml
===mypy quarantine size===
14
```

40. assistant (2026-06-21T13:36:26.458Z)

No automated license-scanning in CI (licensing is enforced by human policy + the de-attribution refusal gate, not a dependency scanner). Now I have a complete picture. Let me do one final verification ? confirm whether the obsreport git+ssh dependency and the auth extra would break a clean `pip install` for a non-repo user, and check the google-genai hard-dependency.

41. user (2026-06-21T13:36:28.007Z)

```
===google-genai hard dep used where===
```

```
backend/providers/gemini.py
===is google-genai lazy?===
6:from google import genai
7:from google.genai import types
```

42. assistant (2026-06-21T13:36:30.614Z)

I now have a complete picture of the verification and licensing constraints. Let me compile my findings.

43. user (2026-06-21T13:37:31.448Z)

Structured output provided successfully

44. assistant (2026-06-21T13:37:40.656Z)

I've completed the investigation and returned my findings via StructuredOutput. Key takeaways on verification and licensing constraints for packaging:

****Verification:**** The suite is fully-offline/mockd (110 test files via `run\_all\_tests.py`), with 5 CI lanes on Python 3.13 (ruff, mypy, import-smoke without GPU stack, full-suite at 70% coverage floor, cross-OS golden fixtures). No built-artifact smoke test exists ? CI tests source only. The import-smoke lane already enforces three packaging-critical boundaries: GPU-stack-absent import, the ADR-008 training/serving import wall, and a pure-numpy/scipy featurizer.

****Licensing:**** Enforced by human policy (MIT/Apache/BSD only for code+data+weights), with NO automated scanner in CI ? the only code-enforced gate is the de-attribution `RefusalError` in the golden-fixtures/training path. The biggest landmine is ffmpeg: current stitch uses GPL libx264; D4 mandates an LGPL build with openh264/hardware encoders.

****Packaging tools:**** PyInstaller (BSD/MIT, GPL-bootloader-with-exception) and Nuitka (Apache) are license-compatible. micromamba (BSD)+conda-forge avoids the Anaconda Distribution ToS that gates conda/Miniconda (already worked around via `conda tos accept` in the Cloud Run Dockerfile). The dual-torch split (2.6+cu124 vs 1.11+cu113) forces a two-env subprocess shape, not a single freeze.

I also surfaced a latent packaging bug: `scipy` is a serving-side runtime dep (`backend/features/rally\_seq.py`) but is undeclared in pyproject ? it only arrives transitively via `supervision`.